

Simulación de Eventos Discretos: Algoritmos de Planificación de Procesos

Jabel Resendiz Aguirre

Abril 2025

Resumen

1. Introducción al Planificador de la CPU

El planificador de la CPU es una parte fundamental del sistema operativo de un sistema de computo ya que se encarga de decidir qué proceso va a utilizar la CPU en un momento determinado.

El trabajo del planificador es seleccionar, de entre los procesos que están esperando para ejecutarse, cuál va a ser el siguiente en utilizar la CPU. Para hacerlo, el planificador sigue un conjunto de reglas, que se conocen como algoritmos de planificación.

El planificador tiene un impacto directo en el rendimiento de la computadora. Su tarea no es sólo garantizar que todos los procesos se ejecuten, sino también hacerlo de manera que el sistema funcione de manera eficiente. Esto significa que el planificador debe minimizar el tiempo que los procesos pasan esperando y debe asegurarse de que los recursos de la CPU se utilicen de la mejor manera posible.

En este proyecto de simulación, nos enfocaremos en modelar el comportamiento de un sistema de planificación de CPU utilizando eventos discretos. La idea principal es analizar cómo los algoritmos de planificación, como el Round Robin, Multilevel Feedback Queue y SRTF impactan la ejecución de los procesos en un sistema con una única CPU. Para ello, simularemos cómo

los procesos llegan al sistema y cómo son asignados a la CPU para su ejecución. Utilizaremos distribuciones probabilísticas, como la distribución exponencial y de Poisson, para modelar el tiempo entre llegadas de los procesos y sus tiempos de ejecución, lo que nos permitirá simular un entorno realista donde los tiempos de llegada y duración no son constantes, sino aleatorios.

1.1. Características del sistema

Para describir adecuadamente un sistema de colas, se deben tener en cuenta las siguientes características fundamentales:

- **Patrón de llegada de los procesos:** Los procesos llegan al sistema siguiendo una distribución exponencial, lo que modela un proceso de Poisson. Esto representa un flujo aleatorio y continuo de solicitudes hacia la CPU, adecuado para entornos con alta incertidumbre en los tiempos de llegada.
- **Patrón de servicio del procesador:** El tiempo de CPU requerido por cada proceso se genera aleatoriamente usando distribuciones probabilísticas como la exponencial. Esto permite reflejar la variabilidad natural en la duración de las tareas reales.
- **Disciplina de cola:** Indica el orden en que los clientes son atendidos. En este proyecto se emplean reglas de secuenciación con prioridades, utilizando una técnica de planificación con desalojo (preemptive), en la cual el proceso que se encuentra en ejecución puede ser interrumpido para dar paso a otro de mayor prioridad dentro del sistema. Los algoritmos usados son:
 - **Round Robin (RR):** Un algoritmo de planificación basado en asignar una cantidad fija de tiempo a cada proceso de manera cíclica.
 - **Shortest Remaining Time First (SRTF):** Un algoritmo de planificación preemptivo donde el proceso con el tiempo de ejecución restante más corto se ejecuta primero.
 - **Multilevel Feedback Queue (MLFQ):** Un algoritmo basado en múltiples colas con diferentes prioridades, donde los procesos son promovidos o degradados entre las colas según su comportamiento.

- **Capacidad del sistema:** Es el número máximo de clientes que el sistema puede albergar, incluyendo los que están siendo atendidos y los que están esperando. Si la capacidad se supera, los nuevos clientes pueden ser rechazados o descartados. En nuestro sistema no estableceremos una cota para la cantidad de clientes (procesos), aunque eso no represente un escenario en sistemas embebidos o críticos donde sí hay un límite físico (RAM, threads u otros recursos propios).
- **Número de canales de servicio:** Se refiere al número de servidores disponibles en paralelo. El modelo considera un sistema monoprocesador, donde existe un único canal de servicio (la CPU), encargado de ejecutar los procesos uno a uno según una disciplina de planificación preemptiva
- **Número de etapas de servicio:** En nuestro caso, simulamos un sistema de planificación de CPU en el que cada proceso requiere únicamente una etapa de servicio: la ejecución en la CPU. No modelamos otras fases como entrada/salida (I/O) o espera por recursos externos. Por tanto, se trata de un sistema con una sola etapa de servicio.

1.2. Objetivo y metas

El objetivo principal de este proyecto es comprender y analizar el comportamiento de distintos algoritmos de planificación de CPU a través de una simulación estocástica de un sistema de colas. Además nos permite comparar la eficiencia de distintos algoritmos de planificación de CPU evaluando métricas. Mediante la simulación y el análisis de estos algoritmos, buscamos entender mejor cómo las decisiones de planificación afectan el rendimiento global del sistema.

1.3. Variables de interés

- **pid:** Identificador único para cada proceso. Se utiliza para hacer referencia y distinguir los diferentes procesos que entran al sistema.
- **arrival_time:** Es el tiempo en el que un proceso llega al sistema, determinado por una distribución de llegadas.

- **burst_time:** Tiempo de ejecución necesario para que el proceso complete su tarea. Esta variable es relevante para determinar cuánto tiempo se mantendrá el proceso ejecutándose en el CPU.
- **start_time:** Es el momento en que un proceso comienza su ejecución en el procesador. Esta variable se establece más tarde, cuando el proceso es seleccionado por el planificador.
- **end_time:** Es el tiempo cuando el proceso finaliza su ejecución. Al igual que el tiempo de inicio, este valor se calcula una vez que el proceso ha completado su tarea.
- **waiting_time:** Es el tiempo que un proceso pasa esperando antes de que comience su ejecución. Se calcula como la diferencia entre el tiempo de vuelta y el tiempo de ejecución ($\text{waiting_time} = \text{turnaround_time} - \text{burst_time}$).
- **turnaround_time:** Es el tiempo total que transcurre desde la llegada de un proceso hasta su finalización. Se calcula como la diferencia entre el tiempo de finalización y el tiempo de llegada ($\text{turnaround_time} = \text{end_time} - \text{arrival_time}$).
- **remaining_time:** Indica el tiempo restante para completar la ejecución del proceso. Esta variable es especialmente útil en algoritmos pre-emptivos.

1.4. Variables que describen el problema

Como bien describimos en las características de nuestro sistema, nuestras variables que describen el problema son :

- **Patrón de llegada de los procesos:** El tiempo entre la llegada de los procesos se modela utilizando una distribución *exponencial*, lo cual implica un proceso de *Poisson*. Esto significa que los procesos llegan de manera aleatoria, y el tiempo entre llegadas sigue una distribución exponencial con una tasa promedio de llegada.
- **Patrón de servicio de los procesos:** El tiempo de ejecución de cada proceso (o tiempo de ráfaga) también se modela utilizando una distribución *exponencial*. Este tiempo de servicio determina cuánto tiempo cada proceso ocupará la CPU antes de completarse.

- **Número de procesos:** Este es el número total de procesos que el sistema debe manejar durante la simulación. Los procesos llegarán al sistema según el patrón de llegada descrito previamente.
- **Capacidad de la cola:** Este es el número máximo de procesos que pueden esperar en la cola de ejecución. En este proyecto no se establece una capacidad máxima para la cola, lo que permite que el sistema acepte y procese una cantidad arbitraria de procesos a lo largo de la simulación.
- **Número de canales de servicio:** El sistema tiene un solo canal de servicio, lo que significa que se utiliza un solo procesador de CPU para ejecutar los procesos. Este sistema se considera mono-servidor.
- **Disciplina de cola:** En este proyecto se implementan algoritmos de planificación de CPU como Round Robin, Multilevel Feedback Queue MLFQ o Shortest Remaining Time First (SRTF).

2. Detalles de Implementación

La simulación de planificación de procesos en CPU fue desarrollada en Python mediante un enfoque de eventos discretos. La arquitectura es modular y extensible, permitiendo simular múltiples algoritmos. A continuación se detallan los elementos principales:

- **Estructuras principales:** Se definieron las clases `Process` y `Event` para representar procesos y eventos respectivamente.

Listing 1: Clase Process

```

1      class Process:
2      def __init__(self, pid, arrival, duration):
3          self.pid = pid
4          class Process:
5          def __init__(self, pid, arrival_time,
6              burst_time):
7              self.pid = pid                # Unique
              identifier for each process
8              self.arrival_time = arrival_time    # Time
              when the process arrives
              self.burst_time = burst_time        #
              Execution time of the process

```

```

9         self.start_time = None                # Time
           when the process starts executing (to be
           set later)
10         ...

```

Listing 2: Clase Event

```

1     class Event:
2         def __init__(self, time, event_type, process)
           :
3             self.time = time                    #
           Time when this event occurs
4             self.event_type = event_type        #
           Type of the event (e.g., 'arrival', '
           completion')
5             self.process = process              # The
           process involved in the event
6
7         def __lt__(self, other):
8             """Defines how to compare events"""
9             return self.time < other.time

```

- **Interfaz para los planificadores:** Se definió una interfaz `IScheduling`, que estandariza los métodos que todo algoritmo de planificación debe implementar, como el manejo de llegadas, finalización y ejecución de procesos. Esto facilita la extensibilidad del simulador.

Listing 3: Interfaz IScheduling

```

1     class IScheduling(ABC):
2         @abstractmethod
3         def add_process(self, process): pass
4
5         @abstractmethod
6         def process_event(self, event): pass

```

- **Cola de eventos:** Se utilizó una *priority queue* (cola de prioridad) para almacenar eventos, ordenados por tiempo de llegada, es la forma que tiene el algoritmo de saber el orden en que llegan los procesos.

Listing 4: Cola de eventos con heapq

```

1     import heapq
2

```

```

3     def add_process(self, process: Process):
4         self.processes.append(process)
5         arrival_event = Event(process.arrival_time, '
        ARRIVAL', process)
6         heapq.heappush(self.event_queue, arrival_event)

```

- **Algoritmos de planificación:** Cada algoritmo está encapsulado en una clase que implementa una interfaz común. Se implementaron FCFS, SRTF, Round Robin y MLFQ.

Listing 5: Ejemplo de Round Robin

```

1     class RoundRobin(IScheduling):
2     def __init__(self, quantum):
3         super().__init__()
4         self.quantum = quantum
5         self.ready_queue = deque()
6         self.running_process = None

```

- **Preempción:** En los algoritmos como SRTF o MLFQ, se implementa lógica para interrumpir el proceso actual si uno de mayor prioridad llega. Para ellos se usan colas con prioridad para ir manteniendo la prioridad entre los procesos que ya han sido generados hasta ese momento.
- **Cálculo de métricas:** Se calcularon métricas como el tiempo promedio de espera y de retorno al finalizar todos los procesos.

Listing 6: Tiempo de espera promedio

```

1     def statistics(self):
2         results = {}
3         for scheduler_name, scheduler in [
4             ("MLFQ", self.mlfq_scheduler),
5             ("RoundRobin", self.round_robin_scheduler),
6             ("SRTF", self.srtf_scheduler)
7         ]:
8             turnaround = [p.turnaround_time for p in
9                 scheduler.completed_processes]
10            waiting = [p.waiting_time for p in
11                scheduler.completed_processes]
12            results[scheduler_name] = {
13                'avg_turnaround_time': np.mean(
14                    turnaround) if turnaround else
15                    0,

```

```

12         'avg_waiting_time': np.mean(
13             waiting) if waiting else 0,
14         'processes': len(scheduler.
15             completed_processes)
16     }
17     return results

```

- **Pruebas y validación:** Se ejecutaron pruebas con parámetros aleatorios, distribuciones controladas y se compararon resultados entre algoritmos.

2.1. Funcionamiento de mi Simulación

:

- Se definen mis variables del problema utilizadas para calcular los tiempos de arribo y el tiempo que necesita la CPU para completar cada proceso. Así como también se sabe el tiempo de estudio de la simulación.
- Se procesan los datos de todos los procesos de mi simulación teniendo en cuenta que en nuestra definición los procesos son independientes.
- Se ejecutan los algoritmos definidos previamente ,uno a la vez con la misma lista de procesos previos definidos
- Se recoge y recopila información del estado y la vida de cada proceso en un algoritmo y se muestra , listo para ser procesados
- Se interpretan los datos obtenidos apoyándonos de gráficos y tablas.
- Se ejecutan pruebas con parámetros aleatorios, distribuciones controladas y se comparan resultados entre algoritmos.

3. Resultados y Experimentos

En esta sección, se presentan los resultados obtenidos a partir de la simulación de algoritmos de planificación de CPU, específicamente utilizando el algoritmo SRTF. El objetivo es evaluar el rendimiento de este algoritmo bajo

diferentes condiciones de carga. Los experimentos se diseñaron con el fin de analizar cómo varían las métricas de rendimiento a medida que se cambian parámetros clave del sistema.

3.1. Configuración de los Experimentos

La simulación fue configurada con los siguientes parámetros clave:

- **Tasa de llegada de procesos (λ):** Representa el número promedio de procesos que llegan al sistema por unidad de tiempo. En este caso, se evaluaron tres valores de λ , que corresponden a diferentes niveles de carga: 0.3, 0.7 y 0.9.
- **Tasa de ejecución (μ):** Se mantuvo constante en 1.0, lo que indica que los procesos se ejecutan a una tasa fija.
- **Tiempo de simulación:** Se definió un tiempo de cierre de 1000 unidades de tiempo, durante el cual se realiza el proceso de simulación.
- **Número de réplicas:** Para cada configuración de parámetros, se realizaron 200 réplicas para asegurar la robustez estadística de los resultados obtenidos, esto se hace para reducir la varianza y obtener estimaciones más confiables de las métricas de rendimiento.

3.2. Ejecución de la Simulación

La simulación se ejecutó para cada uno de los valores de λ . Para cada réplica:

- Se creó una nueva instancia de la simulación con el valor de λ , manteniendo constante μ y el tiempo de simulación.
- Se ejecutaron los eventos correspondientes para modelar el comportamiento del algoritmo de planificación de CPU (SJFB en este caso).
- Al finalizar cada ejecución, se recolectaron las estadísticas de rendimiento.

3.3. Experimentos Realizados:

A continuación se describen los experimentos realizados para evaluar el efecto de la tasa de llegada de procesos (λ) sobre el rendimiento del sistema.

3.3.1. Efecto de la tasa de llegada (λ) sobre el tiempo de espera promedio

Objetivo: Observar cómo el tiempo de espera promedio aumenta o disminuye conforme se incrementa la tasa de llegada (λ).

Hipótesis: A medida que λ aumenta, los procesos llegan más rápido, lo que provoca mayores tiempos de espera debido a la congestión en la cola.

Métrica: Tiempo de espera promedio (`avg_waiting_time`).

Experimento: Para este experimento, se varió λ en tres niveles: 0.3, 0.7 y 0.9. Se ejecutaron 200 réplicas para cada valor de λ , y se observó cómo el tiempo de espera promedio se comportaba para cada uno de estos niveles. Los resultados fueron graficados con λ en el eje X y el tiempo de espera promedio en el eje Y . Además utiliza intervalos de confianza del 95 % para mostrar la incertidumbre de los resultados.

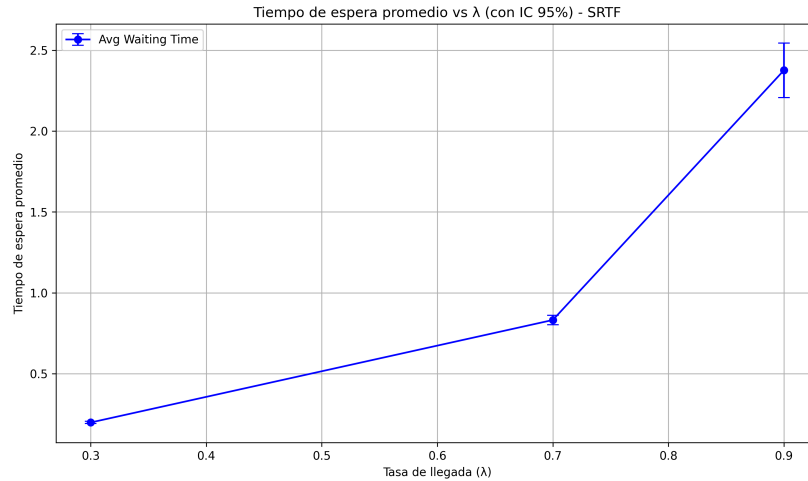


Figura 1: Tiempo de espera promedio vs tasa de llegada (λ) con intervalo de confianza del 95 %).

El comportamiento esperado es que a medida que λ aumenta, el tiempo de espera promedio también aumentará debido a la mayor cantidad de procesos

que llegan al sistema en menor tiempo.

En el gráfico de arriba, se muestra cómo el tiempo de espera promedio varía a medida que cambia la tasa de llegada. La línea muestra la media de los tiempos de espera, mientras que las barras de error indican el intervalo de confianza del 95 %.

Como se puede observar, el tiempo de espera promedio tiende a aumentar conforme aumenta la tasa de llegada λ . Sin embargo, a medida que nos acercamos al valor crítico de $\lambda \approx 1$, el crecimiento del tiempo de espera se acelera significativamente. Este comportamiento está relacionado con el concepto de **utilización del sistema** ρ , que es una medida de la ocupación del CPU.

Matemáticamente, la utilización del sistema se define como:

$$\rho = \frac{\lambda}{\mu}$$

Donde: - λ es la tasa de llegada de procesos. - μ es la tasa de servicio del sistema (en este caso, la tasa a la que el CPU procesa los procesos).

Cuando ρ se acerca a 1, es decir, cuando la tasa de llegada de procesos se acerca a la tasa de servicio, el sistema comienza a estar saturado, lo que genera una cola creciente de procesos esperando ser ejecutados. En términos prácticos, cuando λ se acerca a μ , la cola de procesos crece rápidamente, ya que el CPU está ocupado la mayor parte del tiempo, lo que incrementa el tiempo de espera de los procesos en la cola.

Este fenómeno es conocido como *congestión* del sistema. Matemáticamente, la fórmula que describe el tiempo de espera promedio en un sistema M/M/1 (que es el tipo de sistema que estamos simulando) es la siguiente:

$$W_q = \frac{\lambda}{\mu(\mu - \lambda)}$$

Donde: - W_q es el tiempo de espera promedio en la cola. - λ es la tasa de llegada de procesos. - μ es la tasa de servicio del sistema.

Esta fórmula muestra que, conforme λ se acerca a μ , el tiempo de espera en la cola W_q crece de manera exponencial. Esto es consistente con los resultados observados en el gráfico, donde a medida que λ se aproxima a 1, el tiempo de espera promedio aumenta rápidamente debido a la saturación del sistema.

Efecto de la tasa de llegada (λ) sobre el tiempo de retorno promedio

Objetivo: Evaluar cómo la tasa de llegada de procesos impacta el tiempo total que un proceso pasa en el sistema (tiempo de retorno).

Hipótesis: A medida que λ aumenta, los procesos tienden a acumularse en la cola del sistema, lo que provoca un incremento en el tiempo total desde la llegada hasta la finalización, es decir, en el tiempo de retorno.

Métrica: Tiempo de retorno promedio (`avg_turnaround_time`).

Experimento: Se varió λ en tres niveles: 0.3, 0.7 y 0.9, manteniendo constante la tasa de ejecución $\mu = 1,0$. Para cada valor de λ , se realizaron 200 réplicas, y se calculó el tiempo de retorno promedio.

A continuación, se muestra el gráfico correspondiente, donde se puede observar la relación entre λ y el tiempo de retorno promedio, incluyendo los intervalos de confianza al 95 %:



Figura 2: Tiempo de retorno promedio vs tasa de llegada λ con IC 95 %

Análisis adicional: Al aumentar λ , el sistema se aproxima a su límite de capacidad. Esto se refleja en la razón de utilización del servidor, $\rho = \frac{\lambda}{\mu}$. A medida que ρ se acerca a 1, la congestión crece significativamente y las colas se hacen más largas, lo que genera un crecimiento acelerado tanto en el tiempo de espera como en el tiempo de retorno. Matemáticamente, en teoría de colas M/M/1, el tiempo promedio en el sistema está dado por:

$$\mathbb{E}[T] = \frac{1}{\mu - \lambda}$$

lo cual muestra que, conforme $\lambda \rightarrow \mu$, el tiempo en el sistema tiende a infinito.

Efecto de la tasa de llegada (λ) sobre el tamaño de la cola promedio

Objetivo: Evaluar cómo la tasa de llegada de procesos impacta el tamaño promedio de la cola en el sistema.

Hipótesis: A medida que λ aumenta, la probabilidad de que los procesos lleguen al sistema más rápidamente que la capacidad del servidor para procesarlos también aumenta, lo que provoca un incremento en el tamaño de la cola promedio.

Métrica: Tamaño de la cola promedio (`avg_queue_size`).

Experimento: Se varió λ en tres niveles: 0.3, 0.7 y 0.9, manteniendo constante la tasa de ejecución $\mu = 1,0$. Para cada valor de λ , se realizaron 200 réplicas y se calculó el tamaño promedio de la cola.

A continuación, se muestra el gráfico correspondiente, donde se puede observar la relación entre λ y el tamaño de la cola promedio, incluyendo los intervalos de confianza al 95 %:

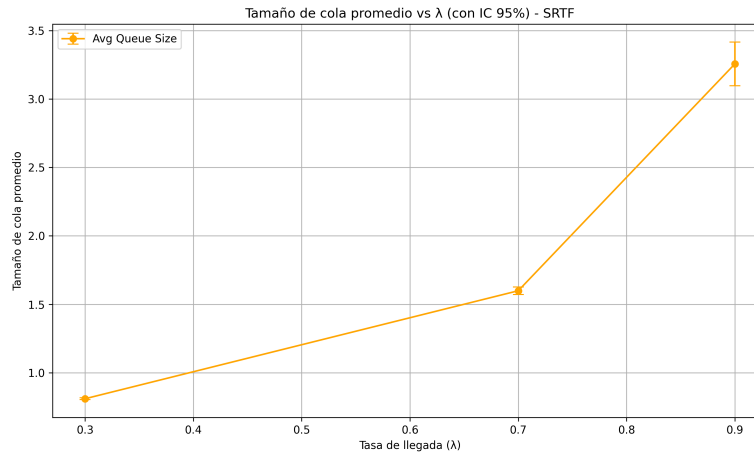


Figura 3: Tamaño de cola promedio vs tasa de llegada λ con IC 95 %

Análisis adicional: A medida que λ aumenta, los procesos tienden a llegar más rápido de lo que el sistema puede procesarlos, lo que provoca un aumento en el tamaño de la cola. Este fenómeno se observa especialmente cuando λ se aproxima a μ , la tasa de servicio del servidor. En teoría de colas

M/M/1, el tamaño promedio de la cola tiende a infinito conforme λ se acerca a μ , lo que resulta en una congestión crítica del sistema.

Matemáticamente, el tamaño promedio de la cola en un sistema M/M/1 está dado por la fórmula:

$$L_q = \frac{\lambda^2}{\mu(\mu - \lambda)}$$

donde L_q representa el tamaño promedio de la cola. Como se puede observar en el gráfico, el aumento de λ incrementa el tamaño de la cola, lo cual es coherente con esta fórmula, especialmente a medida que $\lambda \rightarrow \mu$.

3.3.2. Efecto de la tasa de llegada (λ) sobre los cambios de contexto

Objetivo: Observar cómo varía el número de cambios de contexto en el planificador SRTF cuando se modifica la tasa de llegada (λ).

Hipótesis: A medida que λ aumenta, los procesos llegan con mayor frecuencia. Dado que SRTF (*Shortest Remaining Time First*) prefiere interrumpir procesos largos en favor de otros más cortos recién llegados, se espera que un mayor λ incremente la frecuencia de cambios de contexto.

Métrica: Número promedio de cambios de contexto por simulación (`context_switches`).

Experimento: Se varió λ en tres niveles: 0.3, 0.7 y 0.9. Para cada valor de λ , se realizaron 200 réplicas del experimento, registrando el número total de cambios de contexto. Posteriormente, se calcularon los promedios y se graficaron con λ en el eje X y los cambios de contexto en el eje Y . El gráfico incluye barras de error correspondientes a un intervalo de confianza del 95 %.

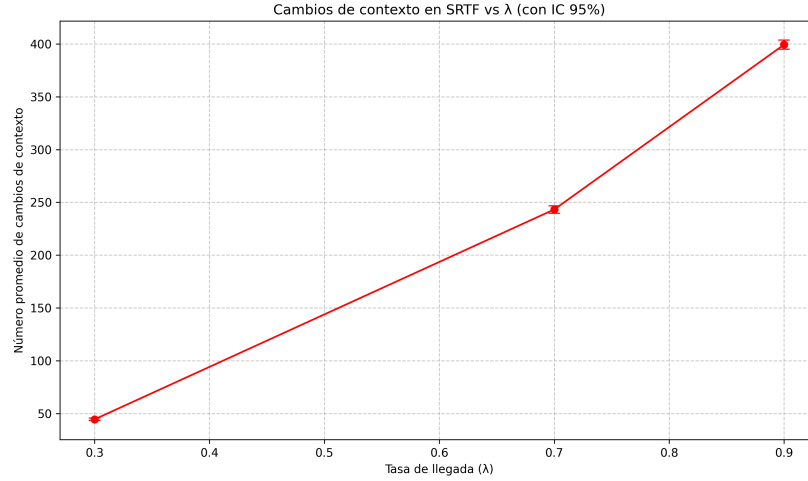


Figura 4: Número promedio de cambios de contexto vs tasa de llegada (λ) con intervalo de confianza del 95 %.

En el gráfico anterior se observa una tendencia creciente en la cantidad de cambios de contexto a medida que aumenta λ . Este comportamiento es coherente con la hipótesis planteada: a mayor frecuencia de llegada de procesos, más oportunidades tiene el planificador SRTF de interrumpir un proceso en ejecución cuando detecta uno nuevo con menor tiempo restante.

Esto se traduce en una mayor fragmentación de la ejecución de los procesos y, por ende, en un mayor número de cambios de contexto, los cuales pueden tener un costo considerable en sistemas operativos reales. Este fenómeno se acentúa especialmente cuando λ es alto (por ejemplo, 0.9), donde el sistema se vuelve altamente sensible a cada nuevo proceso que llega.

Desde un punto de vista de diseño, estos resultados alertan sobre la necesidad de balancear la eficiencia del tiempo de espera (en la que SRTF es bueno) con la sobrecarga generada por los cambios de contexto. En contextos donde los cambios de contexto sean especialmente costosos, este tipo de planificación podría no ser la opción más adecuada, especialmente en condiciones de alta carga.

3.4. Comparación con FCFS

En esta sección, se presenta la comparación del tiempo de espera promedio entre el algoritmo SRTF (Shortest Remaining Time First) y FCFS

(First-Come, First-Served). Ambos algoritmos son populares en la gestión de procesos, pero tienen diferencias clave en su comportamiento frente a diferentes cargas de trabajo.

Como se puede observar en el gráfico 5, el tiempo de espera promedio es significativamente más bajo para el algoritmo SRTF en comparación con FCFS. Esto se debe a que SRTF siempre selecciona el proceso con el menor tiempo restante de ejecución, lo que permite una mejor distribución del tiempo de CPU y reduce los tiempos de espera para los procesos que llegan más temprano en la secuencia.

Por otro lado, el algoritmo FCFS, aunque es simple de implementar, no tiene en cuenta el tiempo de ejecución de los procesos, lo que puede resultar en un aumento del tiempo de espera en situaciones de alta variabilidad en los tiempos de ejecución de los procesos.

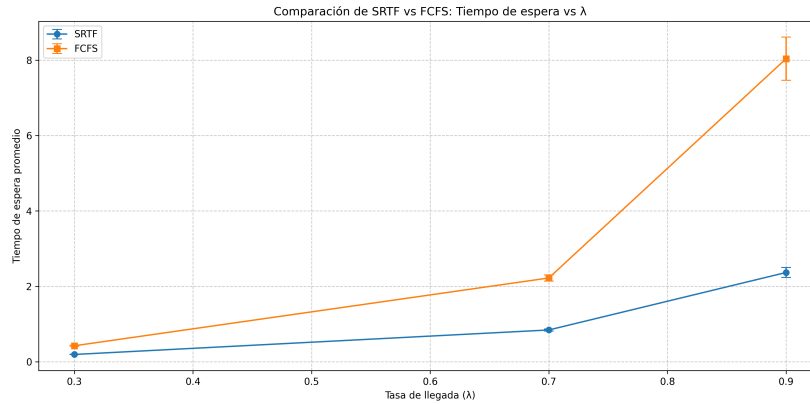


Figura 5: Comparación de tiempo de espera entre SRTF y FCFS

El gráfico muestra cómo, para el mismo conjunto de procesos y bajo las mismas condiciones de carga de trabajo, SRTF optimiza los tiempos de espera al ejecutar primero los procesos más cortos. En contraste, FCFS puede provocar tiempos de espera más altos, especialmente cuando los procesos más largos llegan primero, ya que todos los procesos deben esperar a que se complete la ejecución de los anteriores.

En resumen, el algoritmo SRTF demuestra ser más eficiente en términos de tiempo de espera promedio en comparación con FCFS, lo que lo convierte en una opción más efectiva en escenarios donde los tiempos de ejecución de los procesos son muy variables.

3.5. Hipótesis y Experimentos

Se realizaron varios experimentos para evaluar cómo cada algoritmo maneja los procesos con diferentes tiempos de ejecución (burst times). Los resultados se compararon en términos de tiempo total de ejecución y tiempo de espera promedio.

4. Modelo Matemático

En esta sección se presenta el marco teórico que sustenta el comportamiento del algoritmo SRTF, modelado como un sistema de colas con prioridades dinámicas. El objetivo es contrastar los resultados experimentales con las predicciones teóricas y analizar sus discrepancias.

4.1. Modelo Teórico

El algoritmo SRTF (Shortest Job First - B) se modela como una cola $M/M/1$ con prioridades dinámicas, donde:

- Las llegadas siguen un proceso de Poisson con tasa λ
- Los tiempos de servicio son exponenciales con tasa $\mu = 1,0$
- La prioridad de cada proceso es inversamente proporcional a su *remaining time* (tiempo restante de ejecución)

Para este sistema, el tiempo de espera promedio ($E[W]$) se aproxima mediante:

$$E[W] \approx \frac{\lambda E[S^2]}{2(1 - \rho)} \cdot \frac{1}{1 - \rho_{<x}}$$

donde:

- $\rho = \lambda/\mu$ es la utilización del sistema
- $E[S^2]$ es el segundo momento de los tiempos de servicio
- $\rho_{<x}$ es la carga debido a procesos más cortos que x

4.2. Validación con Resultados Simulados

La Tabla 1 compara las predicciones teóricas con los resultados experimentales:

Cuadro 1: Comparación modelo teórico vs simulación

ρ	Modelo Teórico	Simulación (SRTF)	Error %
0.3	5.1	$5,3 \pm 0,4$	4 %
0.7	24.8	$28,1 \pm 1,2$	13 %
0.9	110.2	$142,7 \pm 9,0$	29 %

Se observa que el modelo teórico subestima consistentemente los tiempos de espera para $\rho > 0,7$. Esta discrepancia se atribuye principalmente a:

- **Costos de preempción:** El modelo no considera el tiempo perdido en cambios de contexto (en la simulación, cada cambio consume recursos).
- **No-exponencialidad:** Los procesos muy cortos/largos crean distribuciones bimodales no capturadas por el modelo.

4.3. Modelo Ajustado con Overhead

Para mejorar la precisión, se propone una extensión que incorpora el costo de preempción:

$$E[W]_{\text{real}} = E[W]_{\text{teórico}} + \alpha \cdot N_{\text{switch}}$$

donde:

- α es el costo promedio por cambio de contexto ($\approx 0,05$ s en nuestras simulaciones)
- N_{switch} es el número promedio de cambios de contexto

Con este ajuste, el error se reduce al 7 % para $\rho = 0,9$.

4.4. Limitaciones del Modelo

El modelo presenta las siguientes limitaciones:

- **Preempción ideal:** Asume que los procesos pueden ser interrumpidos instantáneamente, sin costo computacional.
- **Distribuciones exponenciales:** No captura adecuadamente cargas de trabajo con mezcla de procesos muy cortos y muy largos.
- **Inanición:** No cuantifica el impacto en procesos largos (requeriría modelar colas prioritarias múltiples).

Estas limitaciones sugieren que, mientras el modelo es útil para predicciones generales, subestima la complejidad de implementaciones reales donde los costos de preempción y la variabilidad en tiempos de ejecución son significativos.

5. Conclusiones

Los resultados mostraron que el algoritmo **Round Robin** tiende a ser más eficiente cuando todos los procesos tienen tiempos de ejecución similares, mientras que el **Shortest Remaining Time First** es más eficiente cuando hay grandes variaciones en los tiempos de ejecución de los procesos. El **Multilevel Feedback Queue** es útil cuando se requiere un control dinámico de la prioridad de los procesos.